

Discrete Planning

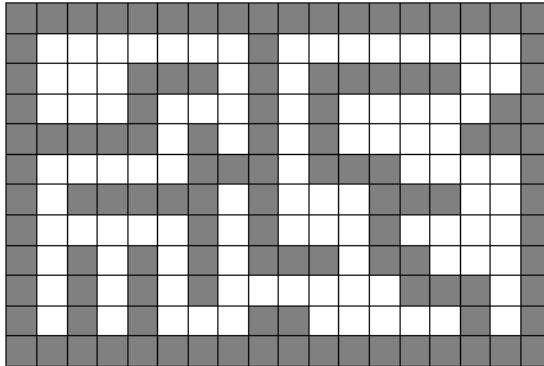
Vikas Dhiman

Monday 8th September, 2025

1 Required readings

- [Ch 7 and 23, Carmen's Intro to Algorithms](#)
- [Breadth-first search and its uses \(article\) | Khan Academy](#)
- [PythonRobotics/dijkstra.py](#)
- [Discrete Planning by Lavalley](#)
- [3.5.2 of Russel and Norvig: Artificial Intelligence](#)

2 Discrete planning in Robotics



2.0.1 Abstraction of a planning problem

1. State space $\mathbf{s} \in \mathcal{S}$. For example, 2D coordinate of a grid $\mathbf{s} = (x, y)$.
2. Action space per state $\mathbf{u} \in \mathcal{U}(\mathbf{s})$. For example, up, down, left right movement can be encoded as $\mathcal{U}(\mathbf{s}_t) = \{(0, -1), (0, 1), (1, 0), (-1, 0)\}$.

3. State transition function $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{u}_t)$. For example, the up-down-left-right action can be combined as addition to get the next state $\mathbf{s}_{t+1} = \mathbf{s}_t + \mathbf{u}_t$.
4. Initial State $\mathbf{s}_I \in \mathcal{S}$
5. Goal states $\mathbf{s}_G \subseteq \mathcal{S}$

2.0.2 A Graph

A graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ is defined by a set of vertices $\mathcal{V}$ and a set of edges $\mathcal{E}$ such that each edge $e \in \mathcal{E}$ is formed by a pair of start and end vertices $e = (v_s, v_e)$, $v_s \in \mathcal{V}$, $v_e \in \mathcal{V}$. The first vertex is called the start of the edge $v_s = \text{start}(e)$ and second vertex is called the end $v_e = \text{end}(e)$.

A discrete planning problem can be converted into a graph by defining

1. Vertices as the state space $\mathcal{V} = \mathcal{S}$.
2. The action space at each state as the edges connected to that vertex/state, $\mathcal{U}(\mathbf{s}_t) = \{(\mathbf{s}_t, \mathbf{s}_j) \mid (\mathbf{s}_t, \mathbf{s}_j) \in \mathcal{E}\}$.
3. State transition function is the other end of the edge, $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{u}_t) = \text{end}(\mathbf{u}_t)$, where $\mathbf{s}_t = \text{start}(\mathbf{u}_t)$.

2.0.3 Representations of Graphs

(Chapter 23 of Introduction to Algorithms by Carmen et al)

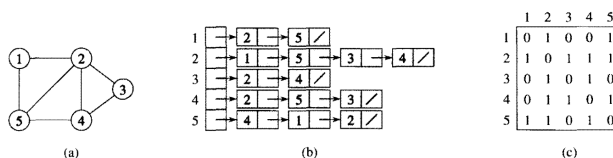


Figure 23.1 Two representations of an undirected graph. (a) An undirected graph G having five vertices and seven edges. (b) An adjacency-list representation of G . (c) The adjacency-matrix representation of G .

Undirected graph

```
# Programmatically you can represent a adjacency list as python lists
# Python lists are not linked lists, they are arrays under the hood.
G_adjacency_list = {
    1 : [2, 5], # node : list of neighbors
    2 : [1, 5, 3, 4],
    3 : [2, 4],
    4 : [2, 5, 3],
    5 : [4, 1, 2]
}
```

Prefer to represent a matrix in python either as a list of lists or a numpy array

```

import numpy as np
G_adjacency_matrix = np.array([
    [0, 1, 0, 0, 1],
    [1, 0, 1, 1, 1],
    [0, 1, 0, 1, 0],
    [0, 1, 1, 0, 1],
    [1, 1, 0, 1, 0]
])

# Edge list is another possible representation
G_edge_list = [
    (1, 2), (1, 5),
    (2, 1), (2, 5), (2, 3), (2, 4),
    (3, 2), (3, 4),
    (4, 2), (4, 5), (4, 3),
    (5, 4), (5, 1), (5, 2)
]

```

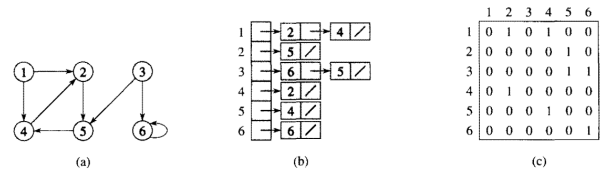


Figure 23.2 Two representations of a directed graph. (a) A directed graph G having six vertices and eight edges. (b) An adjacency-list representation of G . (c) The adjacency-matrix representation of G .

Directed graph representation

```

# Programmatically you can represent a adjacency list as python lists
# Python lists are not linked lists, they are arrays under the hood.
G_adjacency_list = {
    1 : [2, 4],
    2 : [5],
    3 : [6, 5],
    4 : [2],
    5 : [4],
    5 : [6]
}

# Prefer to represent a matrix in python either as a list of lists or a numpy array
import numpy as np
G_adjacency_matrix = np.array([
    [0, 1, 0, 1, 0, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 1, 1],
    [0, 1, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 0],

```

```

        [0, 0, 0, 0, 0, 1]
    ])

# Edge list is another possible representation
G_edge_list = [
    (1, 2), (1, 4),
    (2, 5),
    (3, 6), (3, 5),
    (4, 2),
    (5, 6)
]

# Exercise 1

# Write a function that converts a graph in adjacency list format to adjacency matrix and vice versa
def adjacency_list_to_matrix(G_adj_list):
    G_adj_mat = None # TODO: Write code to convert to adj_mat
    return G_adj_mat

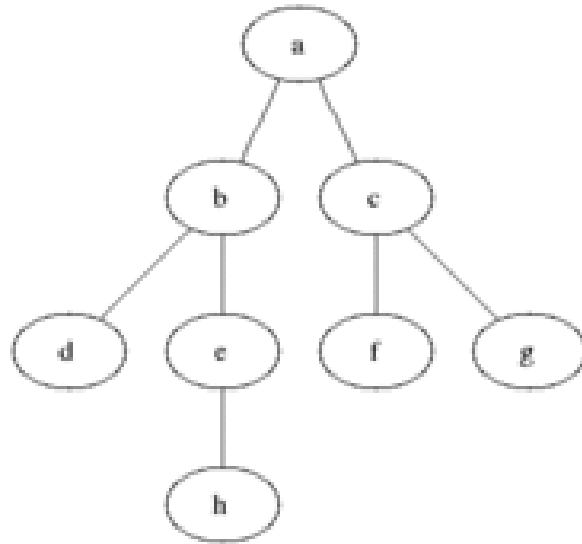
def adjacency_matrix_to_list(G_adj_mat):
    G_adj_list = None # TODO: Write code to convert to adj_list
    return G_adj_list

# Use the above graphs to test
print(adjacency_list_to_matrix(G_adjacency_list))
print(adjacency_matrix_to_list(G_adjacency_matrix))

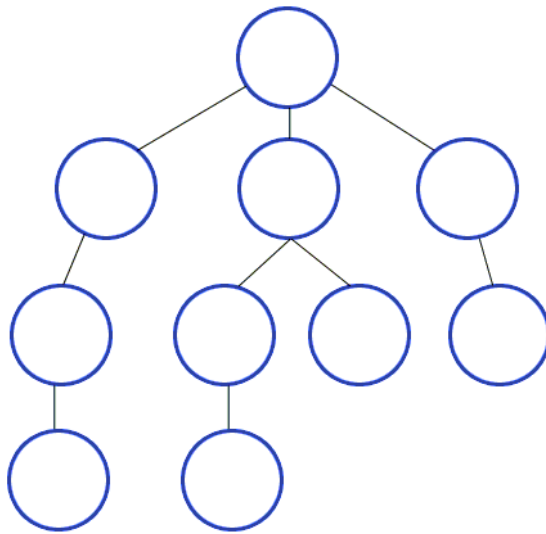
None
None

```

2.0.4 Graph Search algorithms



1. Breadth First Search
2. Depth First Search



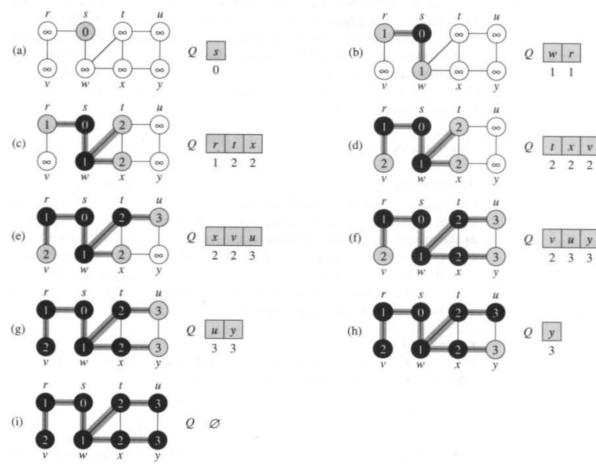


Figure 23.3 The operation of BFS on an undirected graph. Tree edges are shown shaded as they are produced by BFS. Within each vertex u is shown $d[u]$. The queue Q is shown at the beginning of each iteration of the while loop of lines 9–18. Vertex distances are shown next to vertices in the queue.

Breadth first search (BFS)

```
from queue import Queue, LifoQueue, PriorityQueue
```

```
graph = {
    's' : ['w', 'r'],
    'r' : ['v'],
    'w' : ['t', 'x'],
    'x' : ['y'],
    't' : ['u'],
    'u' : ['y']
}

def bfs(graph, start, debug=False):
    seen = set() # Set for seen nodes (contains both frontier and dead states)
    # Frontier is the boundary between seen and unseen (Also called the alive states)
    frontier = Queue() # Frontier of unvisited nodes as FIFO
    node2dist = {start : 0} # Keep track of distances
    search_order = []
    seen.add(start)
    frontier.put(start)

    i = 0 # step number
    while not frontier.empty():
        # Creating loop to visit each node
        if debug: print("%d) Q = " % i, list(frontier.queue), end='; ')
        if debug: print("dists = " , [node2dist[n] for n in frontier.queue])
        m = frontier.get() # Get the oldest addition to frontier
        search_order.append(m)
```

```

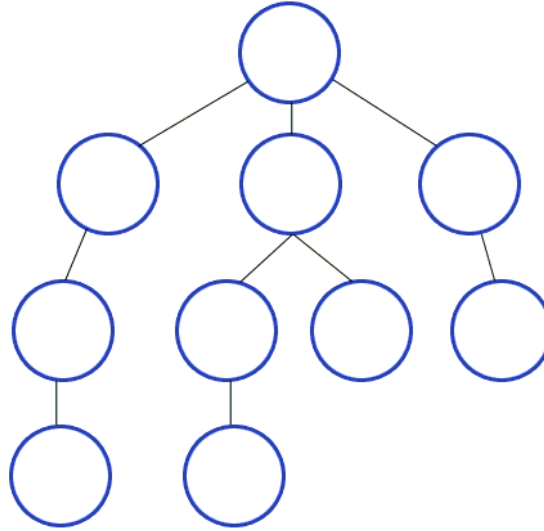
        for neighbor in graph.get(m, []):
            if neighbor not in seen:
                seen.add(neighbor)
                frontier.put(neighbor)
                node2dist[neighbor] = node2dist[m] + 1
            else:
                assert node2dist[neighbor] <= node2dist[m] + 1, 'this should not happen in BFS'
                node2dist[neighbor] = min(node2dist[neighbor], node2dist[m] + 1)

        i += 1
    if debug: print("%d) Q = " % i, list(frontier.queue))
    return search_order, node2dist

print("Following is the Breadth -First Search order")
print(bfs(graph, 's', debug=True))    # function calling

Following is the Breadth -First Search order
0) Q = ['s']; dists = [0]
1) Q = ['w', 'r']; dists = [1, 1]
2) Q = ['r', 't', 'x']; dists = [1, 2, 2]
3) Q = ['t', 'x', 'v']; dists = [2, 2, 2]
4) Q = ['x', 'v', 'u']; dists = [2, 2, 3]
5) Q = ['v', 'u', 'y']; dists = [2, 3, 3]
6) Q = ['u', 'y']; dists = [3, 3]
7) Q = ['y']; dists = [3]
8) Q = []
(['s', 'w', 'r', 't', 'x', 'v', 'u', 'y'], {'s': 0, 'w': 1, 'r': 1, 't': 2, 'x': 2, 'v': 2,

```



Depth first search

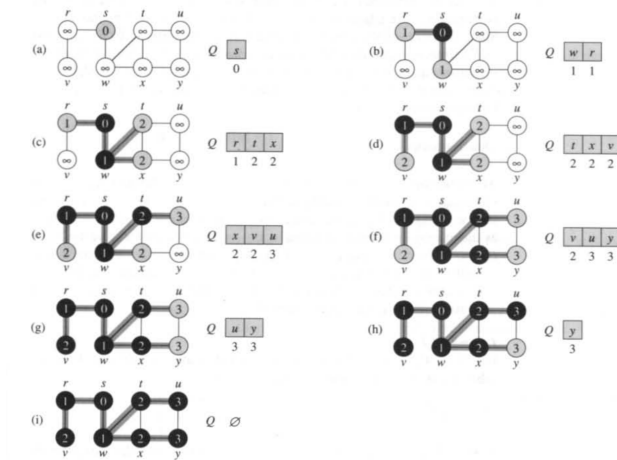


Figure 23.3 The operation of BFS on an undirected graph. Tree edges are shown shaded as they are produced by BFS. Within each vertex u is shown $d[u]$. The queue Q is shown at the beginning of each iteration of the **while** loop of lines 9–18. Vertex distances are shown next to vertices in the queue.

```
graph = {
    's' : ['w', 'r'],
    'r' : ['v'],
    'w' : ['t', 'x'],
    'x' : ['y'],
    't' : ['u'],
    'u' : ['y']
}
```

```

}

def dfs(graph, start, debug=False):
    seen = set([start]) # List for seen nodes (contains both frontier and dead states)
    # Frontier is the boundary between seen and unseen (Also called the alive states)
    frontier = LifoQueue() # Frontier of unvisited nodes as FIFO
    node2dist = {start : 0} # Keep track of distances
    search_order = [] # Keep track of search order
    frontier.put(start)

    i = 0 # step number
    while not frontier.empty():
        # Creating loop to visit each node
        if debug: print("%d) Q = " % i, list(frontier.queue), end='; ')
        if debug: print("dists = " , [node2dist[n] for n in frontier.queue])
        m = frontier.get() # Get the oldest addition to frontier
        search_order.append(m)

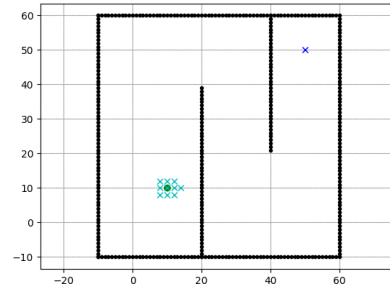
        for neighbor in graph.get(m, []):
            if neighbor not in seen:
                seen.add(neighbor)
                frontier.put(neighbor)
                node2dist[neighbor] = node2dist[m] + 1
            else:
                node2dist[neighbor] = min(node2dist[neighbor], node2dist[m] + 1)
        i += 1
    if debug: print("%d) Q = " % i, list(frontier.queue))
    return search_order, node2dist

# Driver Code
print("Following is the Depth -First Search path")
print(dfs(graph, 's', debug=True)) # function calling

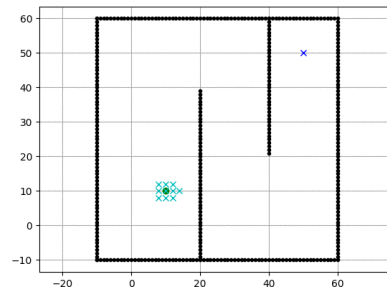
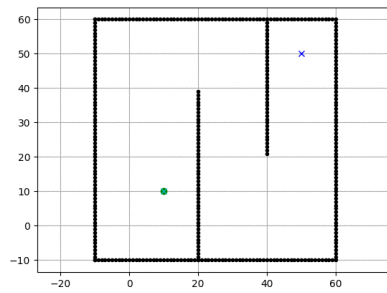
Following is the Depth -First Search path
0) Q = ['s']; dists = [0]
1) Q = ['w', 'r']; dists = [1, 1]
2) Q = ['w', 'v']; dists = [1, 2]
3) Q = ['w']; dists = [1]
4) Q = ['t', 'x']; dists = [2, 2]
5) Q = ['t', 'y']; dists = [2, 3]
6) Q = ['t']; dists = [2]
7) Q = ['u']; dists = [3]
8) Q = []
(['s', 'r', 'v', 'w', 'x', 'y', 't', 'u'], {'s': 0, 'w': 1, 'r': 1, 'v': 2, 't': 2, 'x': 2,

```

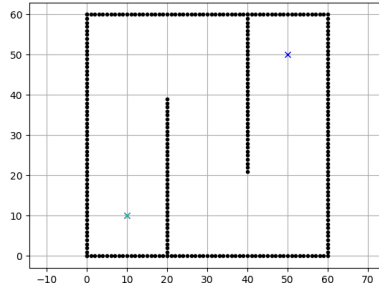
2.0.5 Search order in BFS vs DFS vs Dijkstra



Breadth first search vs Depth first search



Breadth first search vs Dijkstra



2.0.6 Converting a maze search to a graph search

```
import matplotlib.pyplot as plt
import numpy as np
maze_str = \
"""
+++++++
  +      +
+ + + +++
+ + +   +
+ + +   +
+ + +++ +
+      + +
+ +++ + +
+      +
+++++++
"""

class Maze:
    def __init__(self, maze_str, freepath=' '):
        self.maze_lines = [l for l in maze_str.split("\n")
                           if len(l)]
        self.FREEPATH = freepath
        self.fig = None

    def get(self, node, default):
        (r, c) = node
        m_row = self.maze_lines[r]
        nbrs = []
        if c - 1 >= 0 and m_row[c - 1] == self.FREEPATH:
            nbrs.append((r, c - 1))
        if c + 1 < len(m_row) and m_row[c + 1] == self.FREEPATH:
            nbrs.append((r, c + 1))
        if r - 1 >= 0 and self.maze_lines[r - 1][c] == self.FREEPATH:
```

```

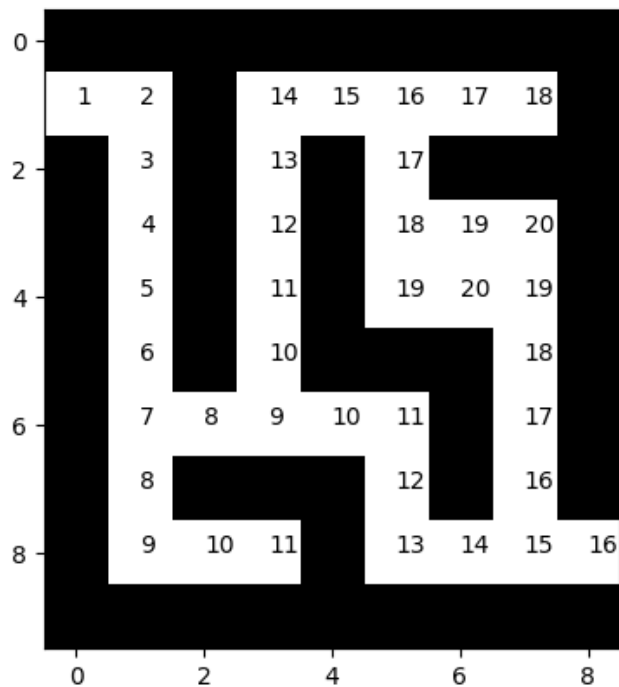
        nbrs.append((r -1, c))
        if r+1 < len(self.maze_lines) and self.maze_lines[r+1][c] == self.FREEPATH:
            nbrs.append((r+1, c))
        return nbrs if len(nbrs) else default
    init_plots = init_plots
    plot_maze = plot_maze
    plot_step = plot_step
    plot_path = plot_path
    animate_search_path = animate_search_path

import matplotlib.pyplot as plt
import matplotlib.animation as animation
import matplotlib as mpl
%matplotlib inline
mpl.rc('animation', html='jshtml')

maze = Maze(maze_str)
search_path, node2dist = bfs(maze, (1, 0)) # prints the order of search all the searched nodes
maze.plot_maze()

maze.animate_search_path(search_path, node2dist)

```



```

def bfs_path(graph, start, goal):
    """
    Returns success and node2parent

    success: True if goal is found otherwise False
    node2parent: A dictionary that contains the nearest parent for node
    """
    seen = [start] # List for seen nodes.
    # Frontier is the boundary between seen and unseen
    frontier = Queue() # Frontier of unvisited nodes as FIFO
    node2parent = dict() # Keep track of nearest parent for each node (requires node to be h
    frontier.put(start)

    while not frontier.empty():          # Creating loop to visit each node
        m = frontier.get() # Get the oldest addition to frontier
        if m == goal:
            return True, node2parent

        for neighbor in graph.get(m, []):
            if neighbor not in seen:
                seen.append(neighbor)
                frontier.put(neighbor)
                node2parent[neighbor] = m
    return False, []

def backtrack_path(node2parent, start, goal):
    c = goal
    r_path = [c]
    parent = node2parent.get(c, None)
    while parent != start:
        r_path.append(parent)
        c = parent
        parent = node2parent.get(c, None) # Keep getting the parent until you reach the start
        #print(parent)
    r_path.append(start)
    return reversed(r_path) # Reverses the path

maze = Maze(maze_str)
start = (1, 0)
goal = (8, 8)
success, node2parent = bfs_path(maze, (1, 0), (8, 8))
path = backtrack_path(node2parent, (1, 0), (8, 8))
#print(list(path))
maze.plot_path(path) # Draws all the searched nodes
plt.show()
#node2parent

```

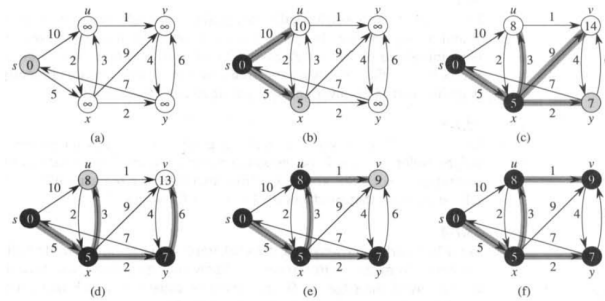
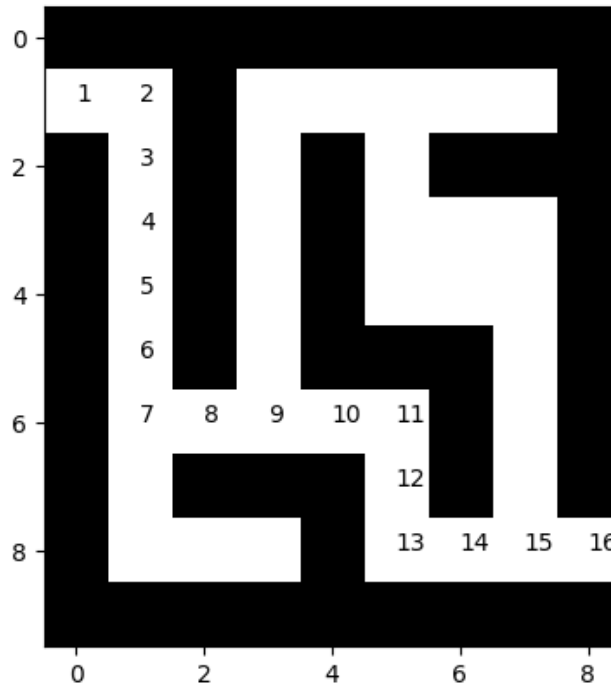


Figure 25.5 The execution of Dijkstra's algorithm. The source is the leftmost vertex. The shortest-path estimates are shown within the vertices, and shaded edges indicate predecessor values: if edge (u, v) is shaded, then $\pi[v] = u$. Black vertices are in the set S , and white vertices are in the priority queue $Q = V - S$. (a) The situation just before the first iteration of the **while** loop of lines 4–8. The shaded vertex has the minimum d value and is chosen as vertex u in line 5. (b)–(f) The situation after each successive iteration of the **while** loop. The shaded vertex in each part is chosen as vertex u in line 5 of the next iteration. The d and π values shown in part (f) are the final values.

Dijkstra algorithm

2.0.7 PriorityQueue

PriorityQueue returns the smallest (or the largest) item in the queue faster than other data structures

```
import itertools
```

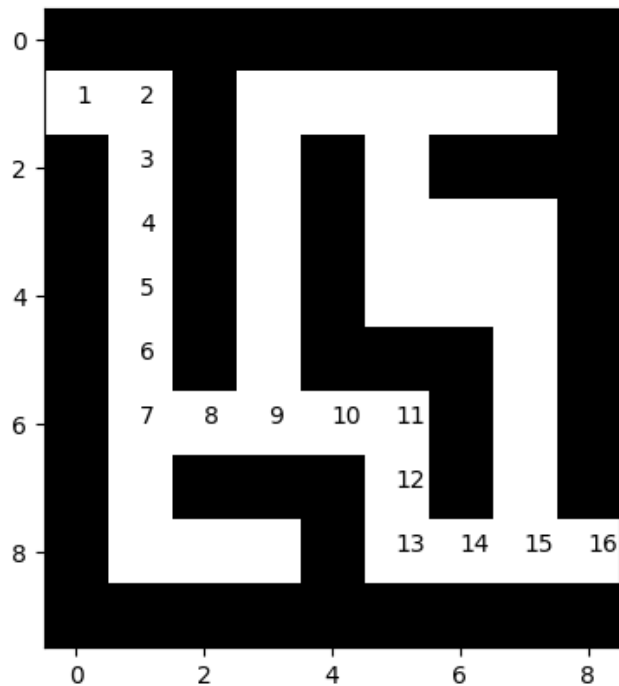
```

class MazeD(Maze):
    def get(self, node, default):
        nbrs = Maze.get(self, node, default)
        return zip(nbrs, itertools.repeat(1))

maze = MazeD(maze_str)
success, search_path, node2parent, node2dist = dijkstra(maze, (1, 0), (8, 8))
print(success, node2parent)
if success:
    path = backtrace_path(node2parent, (1, 0), (8, 8))
    maze.plot_path(path) # Draws all the searched nodes

True {(1, 0): None, (1, 1): (1, 0), (2, 1): (1, 1), (3, 1): (2, 1), (4, 1): (3, 1), (5, 1):

```

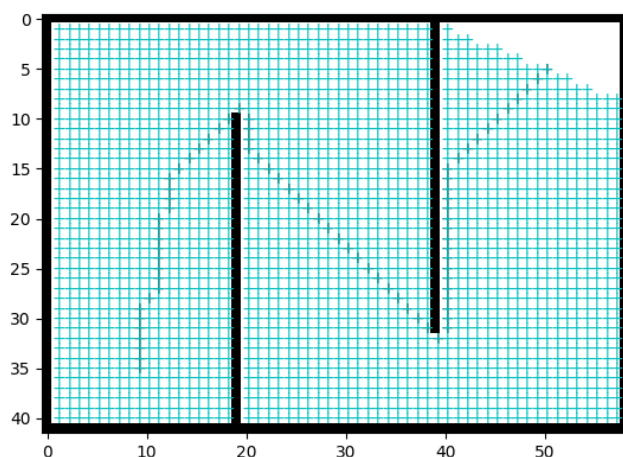


```

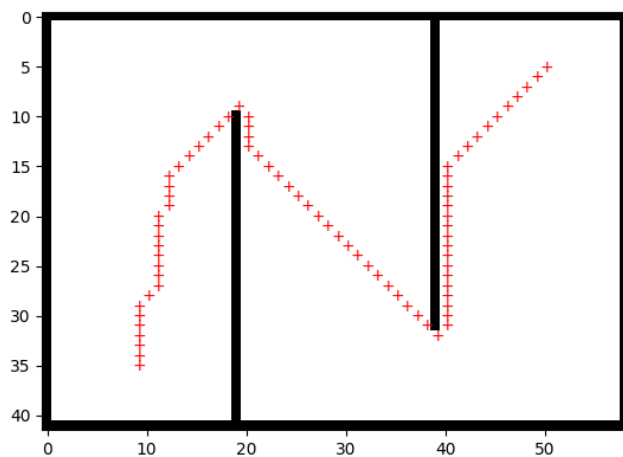
maze = Maze8(maze_str)
success, search_path, node2parent, node2dist = dijkstra(maze, start_pos, goal_pos)
#print(success, search_path)
assert success
anim = maze.animate(search_path)
path = backtrace_path(node2parent, start_pos, goal_pos)
#maze.init_plots(reinit=True)
path_plot = maze.plot_path(path, color='k') # Draws the traced shortest path
anim

```

```
/tmp/ipykernel_3055857/955263672.py:37: UserWarning: frames=<generator object batched at 0x...
anim = animation.FuncAnimation(self.fig, self._plot_path,
```



```
path = backtrace_path(node2parent, (35, 9), (5, 50))
maze.init_plots(reinit=True)
maze.plot_path(path, color='r') # Draws the traced shortest path
plt.show()
```

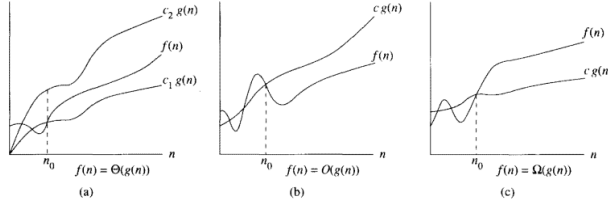


2.1 Computational complexity notation

Asymptotically tight bound $\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_o \text{ such that } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_o\}$

Asymptotic upper bound $O(g(n)) = \{f(n) : \text{there exist positive constants } c_2, \text{ and } n_o \text{ such that } 0 \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_o\}$

Asymptotic lower bound $\Sigma(g(n)) = \{f(n) : \text{there exist positive constants } c_1, \text{ and } n_o \text{ such that } 0 \leq c_1 g(n) \leq f(n) \text{ for all } n \geq n_o\}$



2.1.1 Computational complexity of BFS

```
# Write down the computational complexity of each line in big -O notation O()
# Assume the graph has |V| nodes and |E| edges
def bfs_barebones(graph, start):
    seen = {start} # Set for seen nodes (contains both frontier and dead states) # O(1)
    # Frontier is the boundary between seen and unseen (Also called the alive states)
    frontier = Queue() # Frontier of unvisited nodes as FIFO # O(1)
    frontier.put(start) # O(1)

    while not frontier.empty(): # Creating loop to visit each node # O(|V|)
        m = frontier.get() # Get the oldest addition to frontier # O(|V| * 1)

        for neighbor in graph.get(m, []): # O(|V| * |E|/|V|) = O(|E|)
            if neighbor not in seen: # O(|E| * 1)
                seen.add(neighbor) # O(|E| * 1)
                frontier.put(neighbor) # O(|E| * 1)

# The computational complexity of BFS is O(|E|). Some books write it as O(|V| + |E|)
# where O(|V|) is the cost of initializing states of different nodes
```

2.1.2 Computational complexity of Dijkstra

```
# Write down the computational complexity of each line in big -O notation O()
# Assume the graph has |V| nodes and |E| edges
def dijkstra_barebones(graph, start):
    seen = {start} # Set for seen nodes (contains both frontier and dead states) # O(1)
    # Frontier is the boundary between seen and unseen (Also called the alive states)
    frontier = PriorityQueue() # Frontier of unvisited nodes as PriorityQueue # O(1)
    frontier.put(PItem(0, start)) # O(1)
    node2dist = {start: 0} # Keep track of cost to arrive at each node # O(1)

    while not frontier.empty(): # Creating loop to visit each node
        dist_and_node = frontier.get() # Get the smallest dist node # O(|V| * log(|V|))
        m_dist = dist_and_node.dist
        m = dist_and_node.node
```

```

    for neighbor, edge_dist in graph.get(m, []): #  $O(|V| * |E|/|V|) = O(|E|)$ 
        if neighbor not in seen: #  $O(|E| * 1)$ 
            seen.add(neighbor) #  $O(|E| * 1)$ 
            frontier.put(neighbor) # #  $O(|E| * \log(1))$  # for fibonacci heap
            node2dist[neighbor] = m_dist + edge_dist #  $O(1)$ 
        elif node2dist[neighbor] > m_dist + edge_dist: #  $O(1)$ 
            node2dist[neighbor] = m_dist + edge_dist #  $O(1)$ 

# The computational complexity of Dijkstra is  $O(|V|\log(|V|) + |E|)$  when implemented
# using a Fibonacci heap based PriorityQueue

```

2.2 PriorityQueue (Heaps Chapter 7 of Carmen's intro to algorithms)

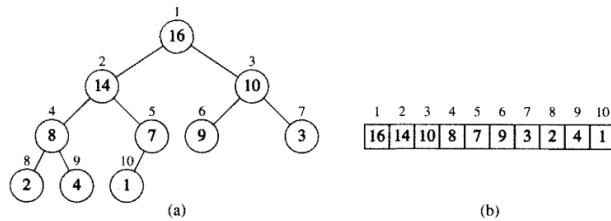


Figure 7.1 A heap viewed as (a) a binary tree and (b) an array. The number within the circle at each node in the tree is the value stored at that node. The number next to a node is the corresponding index in the array.

Heap property

1. $H[\text{Parent}(i)] \geq H[i]$
2. $\text{Parent}(i) = \text{ceil}(i/2)$
3. $\text{LeftChild}(i) = 2i$
4. $\text{RightChild}(i) = 2i+1$

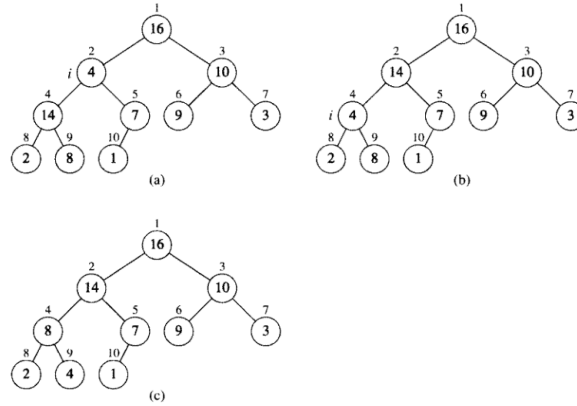


Figure 7.2 The action of $\text{HEAPIFY}(A, 2)$, where $\text{heap-size}[A] = 10$. (a) The initial configuration of the heap, with $A[2]$ at node $i = 2$ violating the heap property since it is not larger than both children. The heap property is restored for node 2 in (b) by exchanging $A[2]$ with $A[4]$, which destroys the heap property for node 4. The recursive call $\text{HEAPIFY}(A, 4)$ now sets $i = 4$. After swapping $A[4]$ with $A[9]$, as shown in (c), node 4 is fixed up, and the recursive call $\text{HEAPIFY}(A, 9)$ yields no further change to the data structure.

Heapify

```

HEAPIFY( $A, i$ )
1   $l \leftarrow \text{LEFT}(i)$ 
2   $r \leftarrow \text{RIGHT}(i)$ 
3  if  $l \leq \text{heap-size}[A]$  and  $A[l] > A[i]$ 
4    then  $\text{largest} \leftarrow l$ 
5    else  $\text{largest} \leftarrow i$ 
6  if  $r \leq \text{heap-size}[A]$  and  $A[r] > A[\text{largest}]$ 
7    then  $\text{largest} \leftarrow r$ 
8  if  $\text{largest} \neq i$ 
9    then exchange  $A[i] \leftrightarrow A[\text{largest}]$ 
10   HEAPIFY( $A, \text{largest}$ )

```

Heapify pseudocode

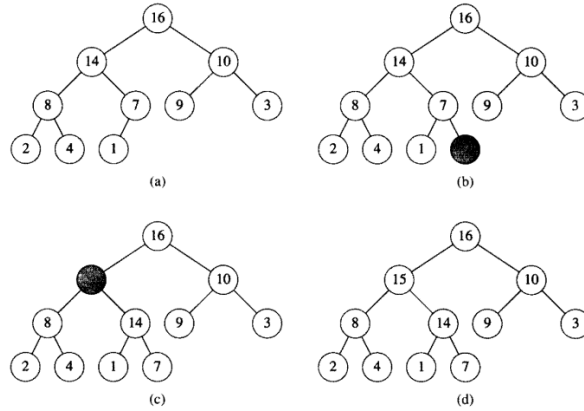


Figure 7.5 The operation of HEAP-INSERT. (a) The heap of Figure 7.4(a) before we insert a node with key 15. (b) A new leaf is added to the tree. (c) Values on the path from the new leaf to the root are copied down until a place for the key 15 is found. (d) The key 15 is inserted.

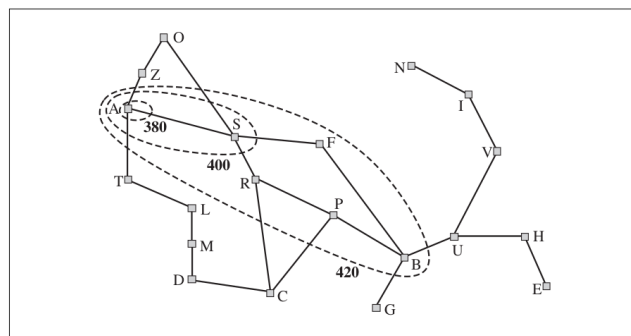
Heap Insert

Operation	find-min	delete-min	insert	decrease-key	meld
Binary ^[9]	$\Theta(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$\Theta(n)$
Leftist	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(\log n)$	$\Theta(\log n)$
Binomial ^{[9][10]}	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)^{[a]}$	$\Theta(\log n)$	$O(\log n)$
Skew binomial ^[11]	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)$	$\Theta(\log n)$	$O(\log n)^{[b]}$
Pairing ^[12]	$\Theta(1)$	$O(\log n)^{[a]}$	$\Theta(1)$	$o(\log n)^{[a][c]}$	$\Theta(1)$
Rank-pairing ^[15]	$\Theta(1)$	$O(\log n)^{[a]}$	$\Theta(1)$	$\Theta(1)^{[a]}$	$\Theta(1)$
Fibonacci ^{[9][2]}	$\Theta(1)$	$O(\log n)^{[a]}$	$\Theta(1)$	$\Theta(1)^{[a]}$	$\Theta(1)$
Strict Fibonacci ^[16]	$\Theta(1)$	$O(\log n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Brodal ^{[17][d]}	$\Theta(1)$	$O(\log n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
2-3 heap ^[19]	$O(\log n)$	$O(\log n)^{[a]}$	$O(\log n)^{[a]}$	$\Theta(1)$	?

Heap runtimes

2.3 A-star (A*) algorithm

(Required reading: 3.5.2 of Russel and Norvig: Artificial Intelligence)



```
from hw2_solution import PriorityQueueUpdatable
```

```

import sys
def astar(graph, heuristic_dist_fn, start, goal, debug=False, debugf=sys.stdout):
    """
    edgecost: cost of traversing each edge

    Returns success and node2parent

    success: True if goal is found otherwise False
    node2parent: A dictionary that contains the nearest parent for node
    """
    seen = set([start]) # Set for seen nodes.
    # Frontier is the boundary between seen and unseen
    frontier = PriorityQueueUpdatable() # Frontier of unvisited nodes as a Priority Queue
    node2parent = {start : None} # Keep track of nearest parent for each node (requires node2parent)
    hfn = heuristic_dist_fn # make the name shorter
    node2dist = {start: 0 } # Keep track of cost to arrive at each node
    search_order = []
    frontier.put(PItem(0 + hfn(start, goal), start)) # < - - - - - Different from dijkstra

    if debug: debugf.write("goal = " + str(goal) + '\n')
    i = 0
    while not frontier.empty():
        # Creating loop to visit each node
        dist_m = frontier.get() # Get the smallest addition to the frontier
        if debug: debugf.write("%d) Q = " % i + str(list(frontier.queue)) + '\n')
        if debug: debugf.write("%d) node = " % i + str(dist_m) + '\n')
        #if debug: print("dists = " , [node2dist[n.node] for n in frontier.queue])
        m = dist_m.node
        m_dist = node2dist[m]
        search_order.append(m)
        if goal is not None and m == goal:
            return True, search_order, node2parent, node2dist

        for neighbor, edge_cost in graph.get(m, []):
            old_dist = node2dist.get(neighbor, float("inf"))
            new_dist = edge_cost + m_dist
            if neighbor not in seen:
                seen.add(neighbor)
                frontier.put(PItem(new_dist + hfn(neighbor, goal), neighbor)) # < - - - - - Different from dijkstra
                node2parent[neighbor] = m
                node2dist[neighbor] = new_dist
            elif new_dist < old_dist:
                node2parent[neighbor] = m
                node2dist[neighbor] = new_dist
            # ideally you would update the dist of this item in the priority queue
            # as well. But python priority queue does not support fast updates
            # - - - - - Different from dijkstra - - - - -

```

```

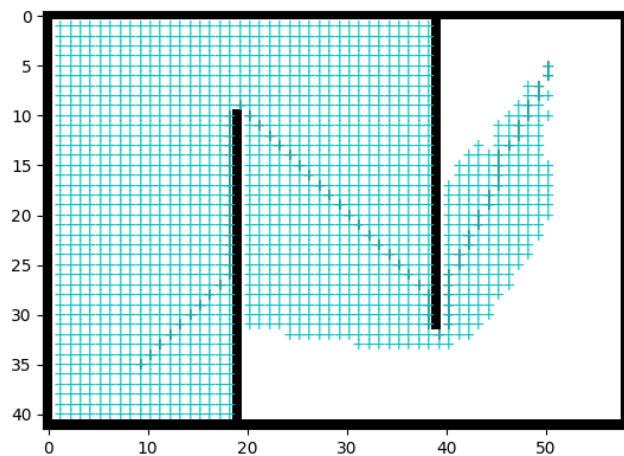
        old_item = PItem(old_dist + hfn(neighbor, goal), neighbor)
        if old_item in frontier:
            frontier.replace(
                old_item,
                PItem(new_dist + hfn(neighbor, goal), neighbor))
        i += 1
    if goal is not None:
        return False, [], {}, node2dist
    else:
        return True, search_order, node2parent, node2dist

import math
from functools import partial

def euclidean_heurist_dist(node, goal, scale=1):
    x_n, y_n = node
    x_g, y_g = goal
    return scale*math.sqrt((x_n - x_g)**2 + (y_n - y_g)**2)

/tmp/ipykernel_3055857/955263672.py:37: UserWarning: frames=<generator object batched at 0x7...
    anim = animation.FuncAnimation(self.fig, self._plot_path,

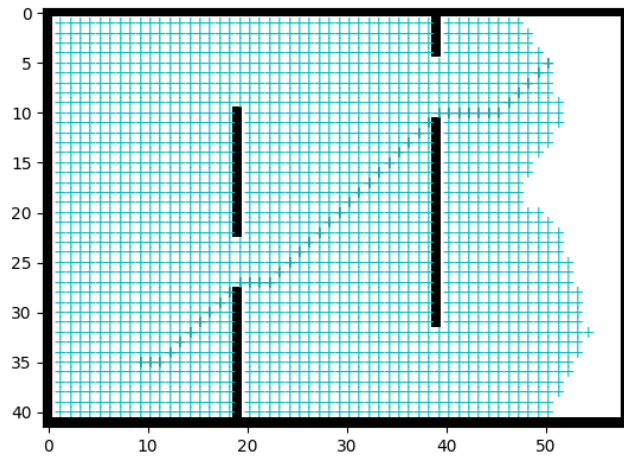
```



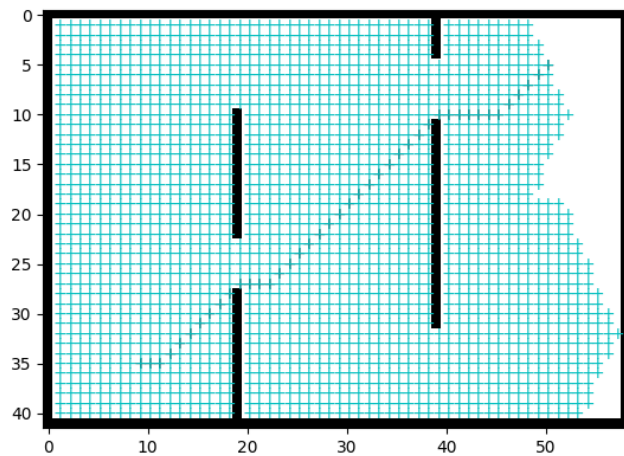
```

/tmp/ipykernel_3055857/955263672.py:37: UserWarning: frames=<generator object batched at 0x7...
    anim = animation.FuncAnimation(self.fig, self._plot_path,

```

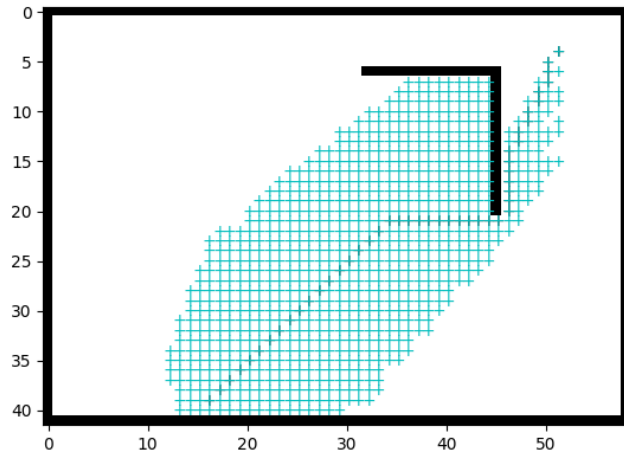


```
/tmp/ipykernel_3055857/955263672.py:37: UserWarning: frames=<generator object batched at 0x7...
  anim = animation.FuncAnimation(self.fig, self._plot_path,
```



```
maze = Maze8(maze_str)
success, search_path, node2parent, node2dist = astar(
    maze, partial(euclidean_heurist_dist, scale=1),
    start_pos, goal_pos)
#print(success, search_path)
assert success
anim = maze.animate(search_path, batch_size=20)
path = backtrace_path(node2parent, start_pos, goal_pos)
#maze.init_plots(reinit=True)
path_plot = maze.plot_path(path, color='k') # Draws the traced shortest path
anim
```

```
/tmp/ipykernel_3055857/955263672.py:37: UserWarning: frames=<generator object batched at 0x...
  anim = animation.FuncAnimation(self.fig, self._plot_path,
```

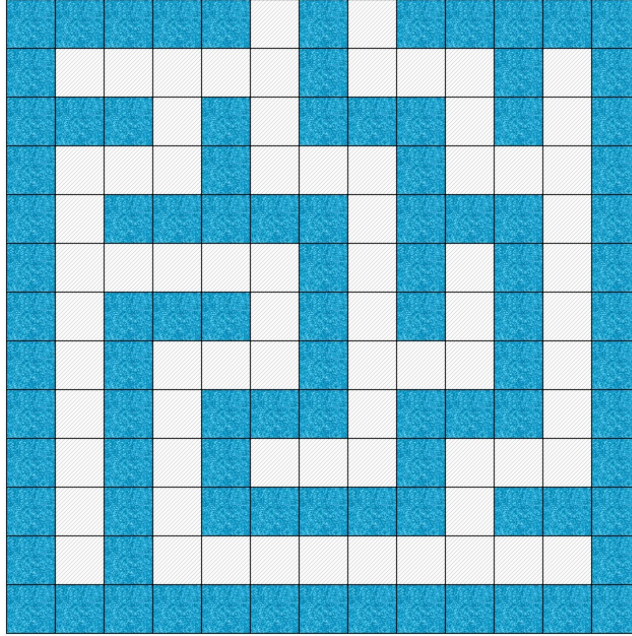


2.3.1 Admisibility and Consistency of heuristic function

1. An admissible heuristic is one that never overestimates the cost to reach the goal.
2. A heuristic $h(n)$ is consistent if it satisfies the triangle inequality:

$$h(s_t) \leq c(s_t, s_{t+1}) + h(s_{t+1}).$$

2.4 Converting maze to grid to graph



2.5 Other ways of converting a maze into a graph

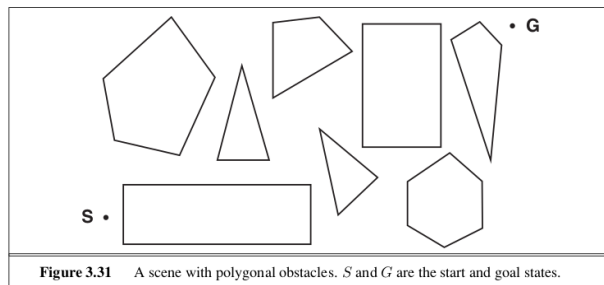


Figure 3.31 A scene with polygonal obstacles. S and G are the start and goal states.

2.6 Rapidly exploring random trees

